

PHY236 Assignment #1: curve fitting in python

Due: 9/4/26 9:00 pm

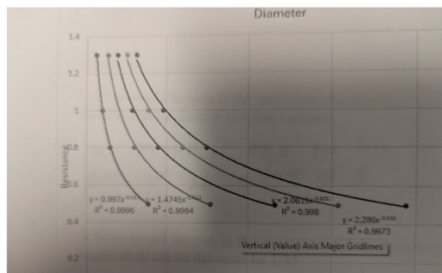
Task #1: Try out curve fitting tool in python:

Down load and upload this into your google Colab notebook:

 Task #1 Curve fitting speedrun

```
import numpy as np
import matplotlib.pyplot as plt # for plot
from scipy.optimize import curve_fit # for curve fitting
from sklearn.metrics import r2_score # to calculate the R-Squared
# run this box to import the library
```

Now re-purpose the codes to do the following curve fitting:



With both **inverse fit** and **inverse square** fit.

Here're the data:

Data for Curve 1

x_data_1 = np.array([1.3, 1.0, 0.8, 0.5])

y_data_1 = np.array([0.60, 0.99, 1.54, 3.84])

Data for Curve 2

x_data_2 = np.array([1.3, 1.0, 0.8, 0.5])

y_data_2 = np.array([1.27, 2.10, 3.23, 7.94])

Data for Curve 3

x_data_3 = np.array([1.3, 1.0, 0.8, 0.5])

y_data_3 = np.array([2.42, 4.01, 6.15, 15.16])

Data for Curve 4

x_data_4 = np.array([1.3, 1.0, 0.8, 0.5])

y_data_4 = np.array([2.99, 4.96, 7.65, 19.02])

To prove your effects, submit a slide with screenshot of crucial steps, and the curve fitting graph generated in python. Then, describe you finding about the curve fitting in a take home message.

Generative AI can provide the boiler plate codes very fast. I am expecting you spend less than **10 min** getting the graph done.

Task 2: Download and work on the cooling curve fitting:

Task #2 cooling curve fitting

Why We Fit Curves in Python and Not in a Spreadsheet

The data and boilerplate code were generated by Claude Opus 5 and modified by xli.

PHY 236 — Advanced Lab I Assignment #1

A cup of coffee was left on a bench with a thermocouple in it. By recording its temperature every 2 minutes for an hour, we can measure the **cooling time constant** — how fast this cup loses heat.

What to turn in: this notebook with all cells run, plus your answers to the two questions at the end.

If you want to **save your own copy**: File -> Save a copy in Drive, and add your name or Coe ID to the file name.

This is the only code block that you need to plug in some code:

```
def cooling(t, T_env, dT, tau):
    return #T000: write out the model function here, with T_env, dT, t and tau. For the exp function, you need to use np.exp.

# T000: rough starting guesses, replacing the following "None" with values for T_env, dT, tau (tau is already filled as 15.0, but feel free to modify the value).
p0 = [None, None, 15.0]

popt, pcov = curve_fit(cooling, t, T, p0=p0, sigma=sigma, absolute_sigma=True)
perr = np.sqrt(np.diag(pcov)) # uncertainty on each fitted parameter

for name, val, err, unit in zip(['T_env', 'dT', 'tau'], popt, perr, ['C', 'C', 'min']):
    print(f"{name:>6} = {val:7.2f} +/- {err:.2f} {unit}")
```

This is the only text box that you need to type your answers:

Questions

Type your answers directly under each one (double-click this cell to edit).

1. Using the table from Part 1, how much did τ change between assuming the room was 18 °C and assuming it was 25 °C? Express it as a percentage of your Python answer.
2. Every one of those spreadsheet fits would have given R-squared above 0.99. In one or two sentences: why is a high R-squared not evidence that an answer is correct?

Before you submit

- ☐ Runtime -> Restart and run all, with no errors
- ☐ `tau` reported with its uncertainty and units
- ☐ The three-curve comparison plot is showing
- ☐ Residual plot shown, and you compared its scatter to the thermocouple uncertainty
- ☐ All two questions answered in your own words

To prove your effects, submit this notebook with all cells run, plus your answers to the two questions at the end.